Chapter 12 ▪ The Java Platform Module System

An important characteristic of object-oriented programming is encapsulation. A class declaration consists of a public interface and a private implementation. A class can evolve by changing the implementation without affecting its users. A module system provides the same benefits for programming in the large. A module can make classes and packages selectively available so that its evolution can be controlled.

Several existing Java module systems rely on class loaders to isolate classes. However, Java 9 introduced a new system, called the Java Platform Module System, that is supported by the Java compiler and virtual machine. It was designed to modularize the large code base of the Java platform. You can, if you choose, use this system to modularize your own applications.

Whether or not you use Java platform modules in your own applications, you may be impacted by the modularized Java platform. This chapter shows you how to declare and use Java platform modules. You will also learn how to migrate your applications to work with the modularized Java platform and third-party modules.

12.1. The Module Concept

In object-oriented programming, the fundamental building block is the class. Classes provide encapsulation. Private features can only be accessed by code that has explicit permission—namely, the methods of the class. This makes it possible to reason about access. If a private variable has changed, you can produce a set of all possible culprits. If you need to modify the private representation, you know which methods are affected.

In Java, packages provide the next larger organizational grouping. A package is a collection of classes. Packages also provide a level of encapsulation. Any feature with package access (neither public nor private) is accessible only from methods in the same package.

However, in large systems, this level of access control is not enough. Any public feature (that is, a feature that is accessible outside a package) is accessible everywhere. Suppose you want to modify or drop a rarely used feature. Once it is public, there is no way to reason about the impact of that change.

This is the situation that the Java platform designers faced. Over twenty years, the JDK grew by leaps and bounds, but clearly some features are now essentially obsolete. Everyone's favorite example is CORBA. When was the last time you used it? Yet, the `org.omg.corba` package was shipped with every JDK until Java 10. As of Java 11, those few who still need it must add the required JAR files to their projects.

What about `java.awt`? It shouldn't be required in a server-side application, right? Except that the class `java.awt.DataFlavor` is used in the implementation of SOAP, an XML-based web services protocol.

The Java platform designers, faced with a giant hairball of code, decided that they needed a structuring mechanism that provides more control. They looked at existing module systems (such as OSGi) and found them unsuitable for their problem. Instead, they designed a new system, called the *Java Platform Module System*, that is now a part of the Java language and virtual machine. That system has been used successfully to modularize the Java API, and you can, if you so choose, use it with your own applications.

A Java platform module consists of

- A collection of packages
- Optionally, resource files and other files such as native libraries
- A list of the accessible packages in the module
- A list of all modules on which this module depends

The Java platform enforces encapsulation and dependencies, both at compile time and in the virtual machine.

Why should you consider using the Java Platform Module System for your own programs instead of following the traditional approach of using JAR files on the class path? There are two advantages.

1. Strong encapsulation: You can control which of your packages are accessible, and you don't have to worry about maintaining code that you didn't intend for public consumption.
2. Reliable configuration: You avoid common class path problems such as duplicate or missing classes.

There are some issues that the Java Platform Module System does not address, such as versioning of modules. There is no support for specifying which version of a module is required, or for using multiple versions of a module in the same program. These can be desirable features, but you must use mechanisms other than the Java Platform Module System if you need them.

12.2. Naming Modules

A module is a collection of packages. The package names in the module need not be related. For example, the module `java.sql` contains packages `java.sql` and `javax.sql`. As you can see from this example, it is perfectly acceptable for the module name to be the same as a package name.

Just like a package name, a module name is made up of letters, digits, underscores, and periods. Also, just as with package names, there is no hierarchical relationship between modules. If you had a module `com.horstmann` and another module `com.horstmann.corejava`, they would be unrelated, as far as the module system is concerned.

When creating a module for use by others, it is important to ensure that its name is globally unique. It is expected that most module names will follow the "reverse domain name" convention, just like package names.

The easiest approach is to name a module after the top-level package that the module provides. For example, the SLF4J logging façade has a module `org.slf4j` with packages `org.slf4j`, `org.slf4j.spi`, `org.slf4j.event`, and `org.slf4j.helpers`.

This convention prevents package name conflicts in modules. Any given package can only be placed in one module. If your module names are unique and your package names start with the module name, then your package names will also be unique.

You can use shorter module names for modules that are not meant to be used by other programmers, such as a module containing an application program. Just to show that it can be done, I will do the same in this chapter. Modules with what could plausibly be library code will have names such as `com.horstmann.util`, and modules containing programs (with a class that has a `main` method) will have catchy names such as `v1ch12.hellomod`.



Note: When referring to a package inside a module, you do not specify the module name. You simply use the package name.

12.3. The Modular “Hello, World!” Program

Let us put the traditional “Hello, World!” program into a module. First, we need to put the class into a package—the “unnamed package” cannot be contained in a module. Here it is:

```
package com.horstmann.hello;

public class HelloWorld {
    public static void main(String[] args) {
        IO.println("Hello, Modular World!");
    }
}
```

So far, nothing has changed. To make a module `v1ch12.hellomod` containing this package, you need to add a module declaration. You place it in a file named `module-info.java`, located in the base directory (that is, the same directory that contains the `com` directory). By convention, the name of the base directory is the same as the module name.

```
v1ch12.hellomod/
└ module-info.java
  com/
  └ horstmann/
    └ hello/
      └ HelloWorld.java
```

The `module-info.java` file contains the module declaration:

```
module v1ch12.hellomod {
}
```

This module declaration is empty because the module has nothing to offer to anyone, nor does it need anything.

Now, compile as usual:

```
javac v1ch12.hellomod/module-info.java \
    v1ch12.hellomod/com/horstmann/hello/HelloWorld.java
```

The `module-info.java` file doesn’t look like a Java source file, and of course there can’t be a class with the name `module-info`, since class names cannot con-

tain hyphens. The `module` keyword, as well as keywords `requires`, `exports`, and so on, that you will see in the following sections, are "restricted keywords" that have a special meaning only in module declarations. The file is compiled into a class file `module-info.class` that contains the module definition in binary form.

To run this program as a modular application, you specify the *module path*, which is similar to the class path but contains modules. You also specify the main class in the format `modulename/classname`:

```
java --module-path v1ch12.hellomod \
    --module v1ch12.hellomod/com.horstmann.hello.HelloWorld
```

Instead of `--module-path` and `--module`, you can use the single-letter options `-p` and `-m`:

```
java -p v1ch12.hellomod -m v1ch12.hellomod/com.horstmann.hello.HelloWorld
```

Either way, the `"Hello, Modular World!"` greeting will appear, demonstrating that you have successfully modularized your first application.



Note: When you compile this module, you get a warning:

```
warning: [module] module name component v1ch12 should avoid terminal digits
```

This warning is intended to discourage programmers from adding version numbers to module names. You can ignore the warning, or suppress it with an annotation:

```
@SuppressWarnings("module")
module v1ch12.hellomod {
}
```

In this one respect, the `module` declaration is just like a class declaration: You can annotate it. (The annotation type must have target `ElementType.MODULE`.)

12.4. Requiring Modules

Let us make a new module `v1ch12.requiremod` in which a program uses a `JOptionPane` to show the "Hello, Modular World!" message:

```
package com.horstmann.hello;

import javax.swing.JOptionPane;

public class HelloWorld {
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(null, "Hello, Modular World!");
    }
}
```

Now compilation fails with this message:

```
error: package javax.swing is not visible
  (package javax.swing is declared in module java.desktop,
  but module v1ch12.requiremod does not read it)
```

The JDK has been modularized, and the `javax.swing` package is now contained in the `java.desktop` module. Our module needs to declare that it relies on that module:

```
module v1ch12.requiremod {
    requires java.desktop;
}
```

It is a design goal of the module system that modules are explicit about their requirements, so the virtual machine can ensure that all requirements are fulfilled before starting a program.

In the preceding section, the need for explicit requirements did not arise because we only used the `java.lang` and `java.io` packages. These packages are included in the `java.base` module which is required by default.

Note that our `v1ch12.requiremod` module lists only its own module requirements. It requires the `java.desktop` module so that it can use the `javax.swing` package. The `java.desktop` module itself declares that it requires three other modules, namely `java.datatransfer`, `java.prefs`, and `java.xml`.

Figure 12.1 shows the *module graph* whose nodes are modules. The edges of the graph (that is, the arrows joining nodes) are either declared requirements or, when no requirement is declared, the implied requirement of `java.base`.

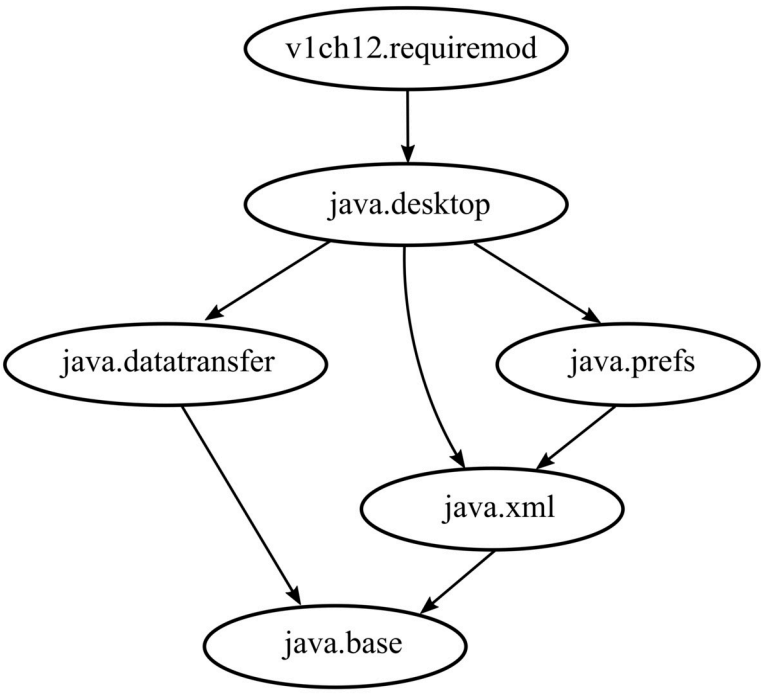


Figure 12.1: The module graph of the Swing "Hello, Modular World" application

You cannot have cycles in the module graph—that is, a module cannot directly or indirectly require itself.

A module does not automatically pass on access rights to other modules. In our example, the `java.desktop` module declares that it requires `java.prefs`, and the `java.prefs` module declares that it requires `java.xml`. That does not give `java.desktop` the right to use packages from the `java.xml` module. It needs to explicitly declare that requirement. In mathematical terms, the `requires` relationship is not "transitive." Generally, this behavior is desirable because it makes requirements explicit, but as you will see in Section 12.11, you can relax it in some cases.



Note: The error message at the beginning of this section stated that our `v1ch12.requiremod` module did not "read" the `java.desktop` module. In the parlance of the Java Platform Module System, module M reads module N in the following cases:

1. M requires N.
2. M requires a module that transitively requires N (see Section 12.11).
3. N is M or `java.base`.

12.5. Exporting Packages

In the preceding section, you saw that a module must require another module if it wants to use its packages. However, that does not automatically make all packages in the required module available. A module states which of its packages are accessible, using the `exports` keyword. For example, here is a part of the module declaration for the `java.xml` module:

```
module java.xml {
    exports javax.xml;
    exports javax.xml.catalog;
    exports javax.xml.datatype;
    exports javax.xml.namespace;
    exports javax.xml.parsers;
    . . .
}
```

This module makes many packages available, but hides others (such as `jdk.xml.internal`) by not exporting them.

When a package is exported, its `public` and `protected` classes and interfaces, and their `public` and `protected` members, are accessible outside the module. (As always, `protected` types and members are accessible only in subclasses and the same package).

However, a package that is not exported is not accessible outside its own module. This is quite different from Java before modules. In the past, you were able to use public classes from any package, even if it was not part of the public API. For example, it was commonly recommended to use classes such as `sun.misc.BASE64Encoder` or `com.sun.rowset.CachedRowSetImpl` when the public API did not provide the appropriate functionality.

Nowadays, you can no longer access unexported packages from the Java platform API since all of them are contained inside modules. As a result, some programs will no longer run with Java 9. Of course, nobody ever committed to keeping non-public APIs available, so this should not come as a shock.

Let us put exports to use in a simple situation. We will prepare a module `com.horstmann.greet` that exports a package, also called `com.horstmann.greet`, following the convention that a module that provides code for others should be named after the top-level package inside it. There is also a package `com.horstmann.greet.internal` that we don't export.

A public `Greeter` interface is in the first package.

```
package com.horstmann.greet;

public interface Greeter {
    static Greeter newInstance() {
        return new com.horstmann.greet.internal.GreeterImpl();
    }

    String greet(String subject);
}
```

The second package has a class that implements the interface. The class is public since it is accessed in the first package.

```
package com.horstmann.greet.internal;

import com.horstmann.greet.Greeter;

public class GreeterImpl implements Greeter {
    public String greet(String subject) {
        return "Hello, " + subject + "!";
    }
}
```

The `com.horstmann.greet` module contains both packages but only exports the first:

```
module com.horstmann.greet {
    exports com.horstmann.greet;
}
```

The second package is inaccessible outside the module.

We put our application into a second module, which will require the first module:

```
module v1ch12.exportedpkg {
    requires com.horstmann.greet;
}
```



Note: The `exports` statement is followed by a package name, whereas `requires` is followed by a module name.

Our application now uses a `Greeter` to obtain a greeting:

```
package com.horstmann.hello;

import com.horstmann.greet.Greeter;

public class HelloWorld {
    public static void main(String[] args) {
        Greeter greeter = Greeter.newInstance();
        IO.println(greeter.greet("Modular World"));
    }
}
```

Here is the source file structure for these two modules:

```
com.horstmann.greet
├ module-info.java
└ com
  └ horstmann
    └ greet
      ├ Greeter.java
      └ internal
        └ GreeterImpl.java
v1ch12.exportedpkg
├ module-info.java
└ com
  └ horstmann
    └ hello
      └ HelloWorld.java
```

To build this application, first compile the `com.horstmann.greet` module:

```
javac com.horstmann.greet/module-info.java \
    com.horstmann.greet/com/horstmann/greet/Greeter.java \
    com.horstmann.greet/com/horstmann/greet/internal/GreeterImpl.java
```

Then compile the application module with the first module on the module path:

```
javac -p com.horstmann.greet v1ch12.exportedpkg/module-info.java \
    v1ch12.exportedpkg/com/horstmann/hello/HelloWorld.java
```

Finally, run the program with both modules on the module path:

```
java -p v1ch12.exportedpkg:com.horstmann.greet \
    -m v1ch12.exportedpkg/com.horstmann.hello.HelloWorld
```

This example demonstrates how an implementation can be hidden in a module. The implementing class is inaccessible outside the module. Clients use an interface and a factory method, instead of instantiating the implementing class.



Note: In the Windows `cmd` shell, you have to enclose `v1ch12.exportedpkg:com.horstmann.greet` in quotation marks because the colon character has a special meaning. You also need to use `^` instead of `\` for multiline commands.



Tip: To build this application with Eclipse, make a separate project for each module. In the `v1ch12.exportedpkg` project, edit the project properties. In the Projects tab, add the `com.horstmann.greet` module to the module path—see **Figure 12.2**.



Figure 12.2: Adding a dependent module to an Eclipse project

You have now seen the `requires` and `exports` statements that form the backbone of the Java Platform Module System. As you can see, the module system is conceptually simple. Modules specify what modules they need, and which packages they offer to other modules. **Section 12.13** shows a minor variation of the `exports` statement.



Caution: A module does not provide a scope. You cannot have two packages with the same name in different modules. This is true even for hidden packages (that is, packages that are not exported.)

12.6. Modular JARs

So far, we have simply compiled modules into the directory tree of the source code. Clearly, that is not satisfactory for deployment. Instead, a module can be deployed by placing all its classes in a JAR file, with a `module-info.class` in the root. Such a JAR file is called a *modular* JAR.

To create a modular JAR file, use the `jar` tool in the usual way. If you have multiple packages, it is best to compile with the `-d` option which places class files into a separate directory. The directory is created if it doesn't already exist. Then use the `-C` option of the `jar` command to change to that directory when collecting files.

```
javac -d modules/com.horstmann.greet \
    com.horstmann.greet/module-info.java \
    com.horstmann.greet/com/horstmann/greet/internal/GreeterImpl.java \
    com.horstmann.greet/com/horstmann/greet/Greeter.java
jar -c -v -f com.horstmann.greet.jar -C modules/com.horstmann.greet .
```

If you use a build tool such as Maven, Ant, or Gradle, just keep building your JAR file as you always do. As long as `module-info.class` is included, you get a modular JAR.

Then, include the modular JAR in the module path, and the module will be loaded.



Caution: In the past, classes of a package were sometimes distributed over multiple JAR files. (Such a package is called a "split package".) This was probably never a good idea, and it is not possible with modules.

As with regular JAR files, you can specify a main class in a modular JAR:

```
javac -p com.horstmann.greet.jar \
    -d modules/v1ch12.exportedpkg \
    v1ch12.exportedpkg/module-info.java \
    v1ch12.exportedpkg/com/horstmann/hello/HelloWorld.java
jar -c -v -f v1ch12.exportedpkg.jar -e com.horstmann.hello.HelloWorld \
    -C modules/v1ch12.exportedpkg .
```

When you launch the program, you specify the module containing the main class:

```
java -p com.horstmann.greet.jar:v1ch12.exportedpkg.jar -m v1ch12.exportedpkg
```

When creating a JAR file, you can optionally specify a version number. Use the `--module-version` argument, and also add the version number to the JAR file name:

```
jar -c -v -f com.horstmann.greet-1.0.jar --module-version 1.0 -C com.horstmann.gree
```

As already discussed, the version number is not used by the Java Platform Module System for resolving modules, but it can be queried by other tools and frameworks.



Note: You can find out the version number through the reflection API. In our example:

```
Optional<String> version = Greeter.class.getModule().getDescriptor().rawVersio
```

yields an `Optional` containing the version string `"1.0"`.



Note: The module equivalent to a class loader is a *layer*. The Java Platform Module System loads the JDK modules and application modules into the *boot layer*. A program can load other modules, using the layer API (which is not covered in this book). Such a program may choose to take module versions into account. It is expected that developers of programs such as Java EE application servers will make use of the layer API to provide support for modules.



Tip: If you want to load a module into JShell, include the JAR on the module path and use the `--add-modules` option:

```
jshell --module-path com.horstmann.greet-1.0.jar --add-modules com.horstmann.g
```

12.7. Modules and Reflective Access

In the preceding sections, you saw that the module system enforces encapsulation. A module can only access explicitly exported packages from another module. In the past, it was always possible to overcome pesky access restrictions by using reflection. As you have seen in **Chapter 5**, reflection can access private members of any class.

However, in the modular world, that is no longer true. If a class is inside a module, reflective access to non-public members will fail. Specifically, recall how we accessed private fields:

```
Field f = obj.getClass().getDeclaredField("salary");
f.setAccessible(true);
double value = f.getDouble(obj);
f.setDouble(obj, value * 1.1);
```

The call `f.setAccessible(true)` succeeds unless a security manager disallows private field access. However, it is not common to run Java applications with security managers, and there are many libraries that use reflective access. Typical examples are object-relational mappers, such as JPA, that automatically persist objects in databases and libraries that convert between objects and XML or JSON, such as JAXB and JSON-B.

If you use such a library, and you also want to use modules, you have to be careful. To demonstrate this issue, let us place the `ObjectAnalyzer` class from **Chapter 5** into a module `com.horstmann.util`. That class has a `toString` method that prints the fields of an object, using reflection.

A separate `v1ch12.openpkg` module contains a simple `Country` class:

```
package com.horstmann.places;

public class Country {
    private String name;
    private double area;

    public Country(String name, double area) {
        this.name = name;
        this.area = area;
    }
    // . . .
}
```

A short program demonstrates how to analyze a `Country` object:

```
package com.horstmann.places;

import com.horstmann.util.*;

public class Demo {
    public static void main(String[] args) throws ReflectiveOperationException {
```

```
        var belgium = new Country("Belgium", 30510);
        var analyzer = new ObjectAnalyzer();
        IO.println(analyzer.toString(belgium));
    }
}
```

Now compile both modules and the `Demo` program:

```
javac com.horstmann.util/module-info.java \
    com.horstmann.util/com/horstmann/util/ObjectAnalyzer.java
javac -p com.horstmann.util v1ch12.openpkg/module-info.java \
    v1ch12.openpkg/com/horstmann/places/*.java
java -p v1ch12.openpkg:com.horstmann.util -m v1ch12.openpkg/com.horstmann.places.De
```

The program will fail with an exception:

```
Exception in thread "main" java.lang.reflect.InaccessibleObjectException:
    Unable to make field private java.lang.String com.horstmann.places.Country.name
    accessible: module v1ch12.openpkg does not "opens com.horstmann.places" to modu
    com.horstmann.util
```

Of course, in pristine theory, it is wrong to violate encapsulation and poke around in the private members of an object. But mechanisms such as object-relational mapping or XML/JSON binding are so common that the module system must accommodate them.

Using the `opens` keyword, a module can *open* a package, which enables reflective access to all instances of classes in the given package. Here is what our module has to do:

```
module v1ch12.openpkg {
    requires com.horstmann.util;
    opens com.horstmann.places;
}
```

With this change, the `ObjectAnalyzer` will work correctly.

A module can be declared as `open`, such as

```
open module v1ch12.openpkg {
    requires com.horstmann.util;
}
```

An open module grants runtime access to all of its packages, as if all packages had been declared with `exports` and `opens`. However, only explicitly exported packages are accessible at compile time. Open modules combine the compile-time safety of the module system with the classic permissive runtime behavior.

Recall from **Chapter 5** that JAR files can contain, in addition to class files and a manifest, *file resources* which can be loaded with the method `Class.getResourceAsStream`, and now also with `Module.getResourceAsStream`. If a resource is stored in a directory that matches a package in a module, then the package must be opened to the caller. Resources in other directories, as well as the class files and manifest, can be read by anyone.



Note: For a more realistic example, we can convert the `Country` object to XML or JSON, using the JSON-B specification. To use the Yasson implementation of JSON-B, download the JAR files `jakarta.json-api-2.1.2.jar`, `jakarta.json.bind-api-3.0.0.jar`, `parsson-1.1.4.jar`, and `yasson-3.0.3.jar` from the Maven Central Repository. Place the JAR files on the module path and run the `com.horstmann.places.Demo` program in the `v1ch12.openpkg2` module. When the `com.horstmann.places` package is opened, conversion to JSON succeeds.



Note: It is possible that future libraries will use *variable handles* instead of reflection for reading and writing fields. A `VarHandle` is similar to a `Field`. You can use it to read or write a specific field of any instance of a specific class. However, to obtain a `VarHandle`, the library code needs a `Lookup` object:

```
public Object getFieldValue(Object obj, String fieldName, Lookup lookup)
        throws NoSuchFieldException, IllegalAccessException {
    Class<?> cl = obj.getClass();
    Field field = cl.getDeclaredField(fieldName);
    VarHandle handle = MethodHandles.privateLookupIn(cl, lookup)
        .unreflectVarHandle(field);
    return handle.get(obj);
}
```

This works provided the `Lookup` object is generated in the module that has the permission to access the field. Some method in the module simply calls `MethodHandles.lookup()`, which yields an object encapsulating the access rights of the caller. In this way, one module can give permission for ac-

cessing private members to another module. The practical issue is how those permissions can be given with a minimum of hassle.

12.8. Automatic Modules

You now know to put the Java Platform Module System to use. If you start with a brand-new project in which you write all the code yourself, you can design modules, declare module dependencies, and package your application into modular JAR files.

However, that is an extremely uncommon scenario. Almost all projects rely on third-party libraries. Of course, you can wait until the providers of all libraries have turned them into modules, and then modularize your own code.

But what if you don't want to wait? The Java Platform Module System provides two mechanisms for crossing the chasm that separates today's premodular world and fully modular applications: automatic modules and the unnamed module.

For migration purposes, you can turn any JAR file into a module simply by placing it onto a directory in the module path instead of the class path. A JAR without a `module-info.class` on the module path is called an *automatic module*. An automatic module has the following properties:

1. The module implicitly has a `requires` clause for all other modules.
2. All of its packages are exported and opened.
3. If there is an entry with key `Automatic-Module-Name` in the JAR file manifest `META-INF/MANIFEST.MF`, its value becomes the module name.
4. Otherwise the module name is obtained from the JAR file name, dropping any trailing version number and replacing sequences of non-alphanumeric characters with a dot.

The first two rules imply that the packages in the automatic module act as if they were on the class path. The reason for using the module path is for the benefit of other modules, allowing them to express dependencies on this module.

Suppose, for example, that you are implementing a module that processes CSV files and uses the Apache Commons CSV library. You would like to express in your `module-info.java` file that your module depends on Apache Commons CSV.

If you add `commons-csv-1.9.0.jar` onto the module path, then your modules can reference the module. Its name is `commons.csv` since the trailing version number `-1.9.0` is removed and the non-alphanumeric character `-` is replaced by a dot.

This name might be an acceptable module name because Commons CSV is well known and it is unlikely that someone else will try to use the same name for a different module. But it would be better if the maintainers of this JAR file could use a name that they control, preferably the top-level package name `org.apache.commons.csv`, as the module name. They just need to add a line

```
Automatic-Module-Name: org.apache.commons.csv
```

to the `META-INF/MANIFEST.MF` file inside the JAR. In fact, they did just that with `commons-csv-1.10.0.jar`.

Eventually, hopefully, they will turn the JAR file into a true module by adding `module-info.java` with that module name. Every other module that refers to the CSV module with that name will just continue to work.



Note: The migration plan to modules is a great social experiment, and nobody knows whether it will end well. Before you put third-party JARs on the module path, check whether they are modular, and if not, whether their manifest has a module name. If not, you can still turn the JAR into an automatic module, but be prepared to update the module name later.

To experiment with automatic JARs, download versions 1.9.0 and 1.10.0 of the Commons CSV library from **https://commons.apache.org/proper/commons-csv**. The `v2ch9.automod` module in the companion code contains a simple program that reads a CSV file with country data:

```
package com.horstmann.places;

import java.io.*;
import org.apache.commons.csv.*;

public class CSVDemo {
    public static void main(String[] args) throws IOException{
        var in = new FileReader("countries.csv");
        Iterable<CSVRecord> records = CSVFormat.EXCEL.withDelimiter(';')
            .withHeader().parse(in);
        for (CSVRecord record : records) {
            String name = record.get("Name");
            double area = Double.parseDouble(record.get("Area"));
            IO.println(name + " has area " + area);
        }
    }
}
```

To use `commons-csv-1.9.0.jar` as an automatic module, we need to require it using its file name:

```
@SuppressWarnings("module")
module v1ch12.automod {
    requires commons.csv;
}
```

For `commons-csv-1.10.0.jar`, use the `Automatic-Module-Name` from the manifest:

```
requires org.apache.commons.csv
```

Here are the commands for compiling and running the program:

```
javac -p v1ch12.automod:jars/commons-csv-1.10.0.jar \
    v1ch12.automod/com/horstmann/places/CSVDemo.java \
    v1ch12.automod/module-info.java
java -p v1ch12.automod:jars/commons-csv-1.10.0.jar \
    -m v1ch12.automod/com.horstmann.places.CSVDemo
```

12.9. The Unnamed Module

Any class that is not on the module path is part of an *unnamed module*. Technically, there may be more than one unnamed module, but all of them together act as if they are a single module which is called *the* unnamed module. As with automatic modules, the unnamed module can access all other modules, and all of its packages are exported and opened.

However, *no explicit module* can access the unnamed module. (An explicit module is a module that is neither automatic nor unnamed—that is, a module with a `module-info.class` on the module path.) In other words, explicit modules are always free from the "class path hell."

Consider, for example, the program of the preceding section. Suppose you put `commons-csv-1.9.0.jar` onto the class path instead of the module path:

```
java --module-path v1ch12.automod \
    --class-path jars/commons-csv-1.10.0.jar \
    -m v1ch12.automod/com.horstmann.places.CSVDemo
```

Now the program won't start:

```
Error occurred during initialization of boot layer
java.lang.module.FindException: Module commons.csv not found, required by v1ch12.au
```

Therefore, migration to the Java Platform Module System is necessarily a bottom-up process:

1. The Java platform itself is modularized.
2. Next, libraries are modularized, either by using automatic modules or by turning them into explicit modules.
3. Once all libraries used by your application are modularized, you can turn the code of your application into a module.



Note: Automatic modules *can* read the unnamed module, so their dependencies can go onto the class path.

12.10. Command-Line Flags for Migration

Even if your programs do not use modules, you cannot escape the modular world when using Java 9 and beyond. Your application code may reside on the class path in an unnamed module, so that all packages are exported and opened. Still, the code interacts with the Java platform, which is modularized.

As of Java 11, compile-time encapsulation is strictly enforced. However, before Java 16, runtime access was permitted. The default behavior was to display a warning on the console for the first instance of each offense. As of Java 16, reflective access at runtime is also enforced. In order to give you time to prepare for that change, the `java` launcher in Java 9 through 16 had an `--illegal-access` flag with four possible settings:

1. `--illegal-access=permit` was the Java 9 default behavior, printing a message for the first instance of illegal access.
2. `--illegal-access=warn` prints a message for each illegal access.
3. `--illegal-access=debug` prints a message and stack trace for each illegal access.
4. `--illegal-access=deny` was the Java 16 default behavior, denying all illegal access.

The `--illegal-access` flag is no longer usable as of Java 17.

The `--add-exports` and `--add-opens` flags allow you to tweak legacy applications. Consider an application that uses an internal API which is no longer accessi-

ble, such as `com.sun.rowset.CachedRowSetImpl`. The best remedy is to change the implementation. (As of Java 7, you can get a cached row set from a `RowSetProvider`.) But suppose you don't have access to the source code.

In that case, start the application with the `--add-exports` flag. Specify the module and the package that you want to export, and the module to which you want to export the package, which in our case is the unnamed module.

```
java --add-exports java.sql.rowset/com.sun.rowset=ALL_UNNAMED \
    -jar MyApp.jar
```

Now, suppose your application uses reflection to access private fields or methods. Reflection inside the unnamed module is OK, but it is no longer possible to reflectively access non-public members of the Java platform classes. For example, some libraries that dynamically generate Java classes call the protected `ClassLoader.defineClass` method through reflection. If an application uses such a library, add the flag

```
--add-opens java.base/java.lang=ALL-UNNAMED
```

When adding all those command-line options to get a legacy app to work, you may well end up with the command line from hell. To better manage multiple options, you can put them in one or more files specified with an `@` prefix. For example,

```
java @options1 @options2 -jar MyProg.java
```

where the files `options1` and `options2` contain options for the `java` command.

There are a few syntax rules for the options files:

- Separate options with spaces, tabs, or newlines.
- Use double quotes around arguments that include spaces, such as `"Program Files"`.
- A line ending in a `\` is merged with the next line.
- Backslashes must be escaped, such as `C:\\Users\\Fred`.
- Comment lines start with `#`.

12.11. Transitive and Static Requirements

In **Section 12.4**, you have seen the basic form of the `requires` statement. In this section, you will see two variants that are occasionally useful.

In some situations, it can be tedious for a user of a given module to declare all required modules. Consider, for example, the `java.desktop` module. It requires three modules: `java.prefs`, `java.datatransfer` and `java.xml`. The `java.prefs` module is only used internally. However, classes from `java.datatransfer` and `java.xml` appear in the public API, in methods such as

```
java.awt.datatransfer.Clipboard java.awt.Toolkit.getSystemClipboard()
java.beans.XMLDecoder(org.xml.sax.InputSource is)
```

That is not something that a user of the `java.desktop` module should have to think about. For that reason, the `java.desktop` module declares the requirement with the `transitive` modifier:

```
module java.desktop {
    requires java.prefs;
    requires transitive java.datatransfer;
    requires transitive java.xml;
    . . .
}
```

Any module that declares a requirement on `java.desktop` now automatically requires these two modules.



Note: Some programmers recommend that you should always use `requires transitive` when a package from another module is used in the public API. But that is not a requirement of the Java language. Consider, for example, the `java.sql` module:

```
module java.sql {
    requires transitive java.logging;
    . . .
}
```

There is a single use of a package from the `java.logging` module in the entire `java.sql` API, namely the `java.sql.Driver.parentLogger` method that returns a `java.util.logging.Logger`. It would have been

perfectly acceptable to not declare this module requirement as transitive. Then, those modules—and only those—who actually use that method would need to declare that they require `java.logging`.

One compelling use of the `requires transitive` statement is an *aggregator* module—a module with no packages and only transitive requirements. One such module is the `java.se` module, declared like this:

```
module java.se {
    requires transitive java.compiler;
    requires transitive java.datatransfer;
    requires transitive java.desktop;
    . . .
    requires transitive java.sql;
    requires transitive java.sql.rowset;
    requires transitive java.xml;
    requires transitive java.xml.crypto;
}
```

A programmer who isn't interested in fine-grained module dependencies can simply require `java.se` and get all modules of the Java SE platform.

Finally, there is an uncommon `requires static` variant that declares that a module must be present at compile time but is optional at runtime. There are two use cases:

1. To access an annotation that is processed at compile time and declared in a different module.
2. To use a class in a different module if it is available, and otherwise do something else, such as:

```
try {
    new oracle.jdbc.driver.OracleDriver();
    . . .
}
catch (NoClassDefFoundError er) {
    Do something else
}
```

12.12. Importing Modules

As you have seen in **Chapter 4**, a variant of the `import` statement lets you import all packages in a module. For example,

```
import module java.desktop;
```

is equivalent to importing more than fifty packages: `java.awt`, `javax.swing`, and so on.

In a compact compilation unit, the `java.base` module is automatically imported.



Note: Prior to Java 25, `jshell` imported an ad-hoc set of packages. Now it imports the `java.base` module.

Of course, when importing that many packages, name clashes become more likely. A number of sample programs in this book import the `java.base` and `java.desktop` modules, each of which has a a `List` class in one of its packages. In order to use the by far more common `java.util.List` without its qualified name, a further `import` statement is required:

```
import java.util.List;
```

You can import an aggregator module. In particular, the statement

```
import module java.se;
```

imports all packages in the Java API. However, that only works in a modular program when a module requires `java.se`. It won't work in a compact compilation unit.

12.13. Qualified Exporting and Opening

In this section, you will see a variant of the `exports` and `opens` statement that narrows their scope to a specified set of modules. For example, the `java.base` module contains a statement

```
exports sun.net to
  java.net.http,
  jdk.naming.dns;
```

Such a statement is called a *qualified export*. The listed modules can access the exported package, but other modules cannot.

Excessive use of qualified exports can indicate a poor modular structure. Nevertheless, they can arise when modularizing an existing code base. Here, the `sun.net` package is placed inside the `java.base` module because that is where it is mostly needed. However, a couple of other modules also use that package. The Java platform designers didn't want to make `java.base` even bigger, and they didn't want to make the internal `sun.net` package generally available. In a greenfield project, one can instead design a more modular API.

Similarly, you can restrict the `opens` statement to specific modules. For example, in **Section 12.7** we could have used a qualified `opens` statement, like this:

```
module v1ch12.openpkg {
    requires com.horstmann.util;
    opens com.horstmann.places to com.horstmann.util;
}
```

Now the `com.horstmann.places` package is only opened to the `com.horstmann.util` module.

12.14. Service Loading

The `ServiceLoader` class (see **Chapter 6**) provides a lightweight mechanism for matching up service interfaces with implementations. The Java Platform Module System makes this mechanism easier to use.

Here is a quick reminder of service loading. A service has an interface and one or more possible implementations. Here is a simple example of an interface:

```
public interface GreeterService {
    String greet(String subject);
    Locale getLocale();
}
```

One or more modules provide implementations, such as

```
public class FrenchGreeter implements GreeterService {
    public String greet(String subject) { return "Bonjour " + subject; }
    public Locale getLocale() { return Locale.FRENCH; }
}
```

The service consumer must pick an implementation among all offered implementations, based on whatever criteria it deems appropriate.

```
ServiceLoader<GreeterService> greeterLoader = ServiceLoader.load(GreeterService.cla
GreeterService chosenGreeter;
for (GreeterService greeter : greeterLoader) {
    if (. . .) {
        chosenGreeter = greeter;
    }
}
```

In the past, implementations were offered by placing text files into the `META-INF/services` directory of the JAR file containing the implementation classes. The module system provides a better approach. Instead of text files, you can add statements to the module descriptors.

A module providing an implementation of a service adds a `provides` statement that lists the service interface (which may be defined in any module) and the implementing classes (which must be a part of this module). Here is an example from the `jdk.security.auth` module:

```
module jdk.security.auth {
    . . .
    provides javax.security.auth.spi.LoginModule with
        com.sun.security.auth.module.Krb5LoginModule,
        com.sun.security.auth.module.UnixLoginModule,
        com.sun.security.auth.module.JndiLoginModule,
        com.sun.security.auth.module.KeyStoreLoginModule,
        com.sun.security.auth.module.LdapLoginModule,
        com.sun.security.auth.module.NTLoginModule;
}
```

A consuming module contains a `uses` statement.

```
module java.base {
    . . .
    uses javax.security.auth.spi.LoginModule;
}
```

When code in a consuming module calls `ServiceLoader.load(`ServiceInterface`.class)`, the matching provider classes will be loaded, even though they may not be in accessible packages.

In our code example, we provide implementations for a German and French greeter in the package `com.horstmann.greetsvc.internal`. The service module exports the `com.horstmann.greetsvc` package, but not the package with

the implementations. The `provides` statement declares the service and its implementing classes in the unexported package:

```
module com.horstmann.greetsvc {
    exports com.horstmann.greetsvc;

    provides com.horstmann.greetsvc.GreeterService with
        com.horstmann.greetsvc.internal.FrenchGreeter,
        com.horstmann.greetsvc.internal.GermanGreeterFactory;
}
```

The `v1ch12.useservice` module consumes the service. Using the `ServiceLoader` facility, we iterate over the provided services and pick the one matching the desired language:

```
package com.horstmann.hello;

import java.util.*;
import com.horstmann.greetsvc.*;

public class HelloWorld {
    public static void main(String[] args) {
        ServiceLoader<GreeterService> greeterLoader
            = ServiceLoader.load(GreeterService.class);
        String desiredLanguage = args.length > 0 ? args[0] : "de";
        GreeterService chosenGreeter = null;
        for (GreeterService greeter : greeterLoader) {
            if (greeter.getLocale().getLanguage().equals(desiredLanguage))
                chosenGreeter = greeter;
        }
        if (chosenGreeter == null)
            IO.println("No suitable greeter.");
        else
            IO.println(chosenGreeter.greet("Modular World"));
    }
}
```

The module declaration requires the service module and declares that the `GreeterService` is being used.

```
module v1ch12.useservice {
    requires com.horstmann.greetsvc;
    uses com.horstmann.greetsvc.GreeterService;
}
```

As a result of the `provides` and `uses` declarations, the module that consumes the service is allowed access to the module-private implementation classes.

To build and run the program, first compile the service:

```
javac com.horstmann.greetsvc/module-info.java \
    com.horstmann.greetsvc/com/horstmann/greetsvc/GreeterService.java \
    com.horstmann.greetsvc/com/horstmann/greetsvc/internal/*.java
```

Then compile and run the consuming module:

```
javac -p com.horstmann.greetsvc \
    v1ch12.useservice/com/horstmann/hello/HelloWorld.java \
    v1ch12.useservice/module-info.java
java -p com.horstmann.greetsvc:v1ch12.useservice \
    -m v1ch12.useservice/com.horstmann.hello.HelloWorld
```

12.15. Tools for Working with Modules

This section covers the `jdeps`, `jlink`, and `jmod` tools that are a part of the Java Development Kit.

The `jdeps` tool analyzes the dependencies of a given set of JAR files. Suppose, for example, that you want to modularize JUnit 4. Run

```
jdeps -s junit-4.12.jar hamcrest-core-1.3.jar
```

The `-s` flag generates a summary output:

```
hamcrest-core-1.3.jar -> java.base
junit-4.12.jar -> hamcrest-core-1.3.jar
junit-4.12.jar -> java.base
junit-4.12.jar -> java.management
```

That tells you the module graph, as shown in **Figure 12.3**.

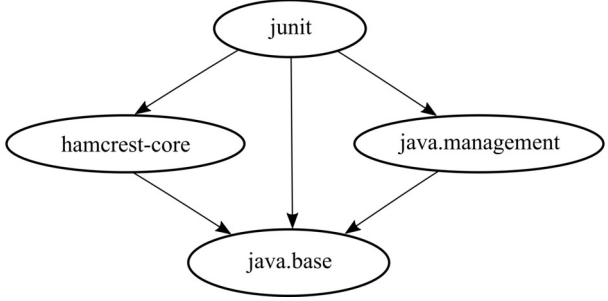


Figure 12.3: The JUnit4 Module Graph

If you omit the `-s` flag, you get the module summary followed by a mapping from packages to required packages and modules. If you add the `-v` flag, the listing maps classes to required packages and modules.

The `--generate-module-info` option produces `module-info` files for each analyzed module:

```
jdeps --generate-module-info /tmp/junit junit-4.12.jar hamcrest-core-1.3.jar
```



Note: There is also an option to generate graphical output in the "dot" language for describing graphs. Assuming you have the dot tool installed, run these commands:

```
jdeps -s -dotoutput /tmp/junit junit-4.12.jar hamcrest-core-1.3.jar
dot -Tpng /tmp/junit/summary.dot > /tmp/junit/summary.png
```

Then `summary.png` looks as in **Figure 12.4**.

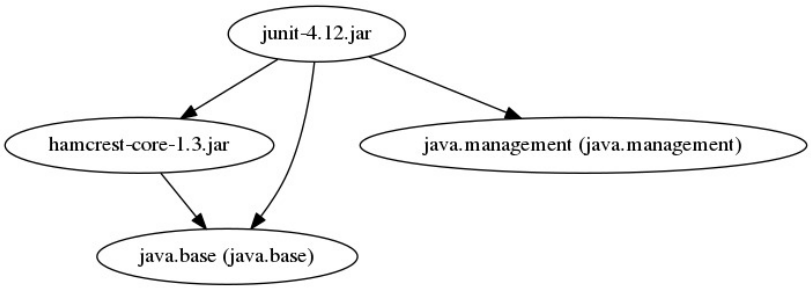


Figure 12.4: The Dot Output of `jdeps`

Use the `jlink` tool to produce an application that executes without a separate Java runtime. The resulting image is much smaller than the entire JDK. You specify the modules that you want to have included and an output directory.

```
jlink --module-path com.horstmann.greet.jar:v1ch12.exportedpkg.jar:$JAVA_HOME/jmods
    --add-modules v1ch12.exportedpkg --output /tmp/hello
```

The output directory has a subdirectory `bin` with a `java` executable. If you run

```
bin/java -m v1ch12.exportedpkg
```

the `main` method of the module's main class is invoked.

The point of `jlink` is that it bundles up the minimal set of modules required to run the application. You can list them all:

```
bin/java --list-modules
```

In this example, the output is

```
v1ch12.exportedpkg
com.horstmann.greet
java.base@25
```

All modules are included in a *runtime image* file `lib/modules`. On my computer, that file is 29MB, whereas the runtime image of all JDK modules takes up 133MB. The entire application takes up 55MB, a fraction of the size of the JDK.

This can be the basis of a useful tool for packaging applications. You would still need to produce file sets for multiple platforms and launch scripts for the application.



Note: You can inspect the runtime image with the `jimage` command. However, the format is internal to the JVM, and runtime images are not meant to be generated or used by other tools.

Finally, the `jmod` tool builds and inspects the module files that are included with the JDK. When you look into the `jmods` directory inside the JDK, you will find a file with extension `jmod` for each module. There is no longer a `rt.jar` file.

Like JAR files, these files contain class files. In addition, they can hold native code libraries, commands, header files, configuration files, and legal notices. The JMOD files use the ZIP format. You can inspect their contents with any ZIP tool.

Unlike JAR files, JMOD files are only useful for linking—that is, for producing run-time images. There is no need for you to produce JMOD files unless you also want to bundle binary files such as native code libraries with your modules.

You have now reached the end of Volume I of *Core Java*. This volume covered the fundamentals of the Java programming language and the parts of the standard library that you need for most programming projects. I hope that you enjoyed your tour through the Java fundamentals and that you found useful information along the way. For advanced topics, such as the stream library, file input and output, networking, security, internationalization, user interfaces, and graphics programming, please turn to Volume II.

Previous chapter
< 11. Annotations

Next chapter
Appendix >